

Storing Answer

- ▶ **Type**
 - ▶ Array
 - ▶ `Vector<string>`
 - ▶ ... of ...
 - ▶ Store index of word
 - ▶ Store actual word
-
- ▶ OR nothing at all, just display

Recursive Backtracking

- ▶ **Base Case**
 - ▶ Bool for bad track
 - ▶ End of array/vector/LL
 - ▶ No letters left to use
- ▶ **Recursion**
 - ▶ Find place to start searching
 - ▶ Comparison of letter's
 - ▶ Comparison of numbers



Duplicates

- ▶ Bool should catch a bad track, however, the array answer will still exist.
- ▶ Just out of curiosity:
Is 'w1 w2 w3' a duplicate with 'w1 w2 w4'???

Big $O(\infty)$

- ▶ **Primary Goal: Shrink (N) fast**
 - ▶ Limit what part of the dictionary I'm looking at.
- ▶ **Secondary Goal: Pick words wisely**
 - ▶ Limit the amount of duplicates to increase speed.

GCC Optimization

COIN 431 – Advanced Operating Systems

Fall 2009

COIN 431 Grade: B

Levels of Optimization

Optimization increases compile time to reduce execution time and increase performance.

-O0

Default optimization, no optimization.

-O1

The goal of O1 is to reduce size and increase speed. It does this by eliminating things like dead code, jumps for size. And increases performance by minimizing the number of times the stack is used and when register values are copied.

-O2

O2 includes all optimizations of O1, however, the new performance should be noticeable. The compiler uses alignment such that functions, jumps, labels, and loops are pushed to line up on the boundaries, powers of 2. Making the access time smaller.

-Os

Special or Size Optimization is quite popular because it incorporates all the optimization of O2, however, turning off the Optimizations known to increase the size of the output.

-O3

Level 3 enables all optimization, trading as much space for performance as it can.

Intermediate Advantages

Originally GCC just translated code to Register Transfer Language.

However, creating more steps gives the compiler more opportunities to tweak and fine-tune your code.

RTL alone is not good with data, because it is hard for the compiler to identify the different variables and data types.

Intermediate Steps

Methods of Optimization

Compilers use several methods to determine how to improve their generated code.

Control Flow Analysis – Examines loops and other control structures, such as if-statements and case statements, to identify the execution paths a program might take and, based on this analysis, determine how execution paths can be streamlined.

Data Flow Analysis – Examines how and when variables are used in a program and then applies various set equations to these usage patterns to derive optimization opportunities.

Optimization occurs in local transformations and global transformations, performing local before global.

Common Subexpression Elimination

Eliminates redundant computations such that it reduces the number of instructions without changing the outcome of the program.

Constant Folding

Eliminates statements that calculate a value that can be determined when compiled instead of execution.

Copy Propagation Transformation

Eliminates the look-up of a variable's value by using the value instead of the variable. Also computed at compile time.

Dead Code Elimination

Eliminates “Unreachable” code.

Code that will never be executed by the program.

This is often caught by the programmer, however, GCC checks to make sure anyways.

If-Conversion

The Compiler looks to see if there is a better way to rearrange long if-statements. This is done to reduce jumps and branches whenever possible.

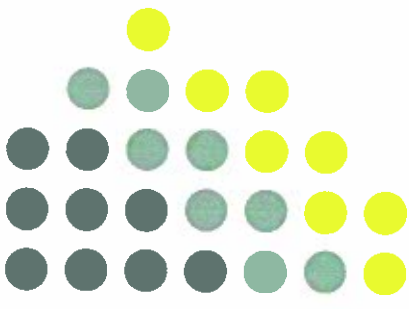
Inlining

Also called “loop unrolling” or “Function Inlining.”

Reduces the complexity of the code by moving the code where functions are called or unraveling a loop into straight execution.

GCC Optimization

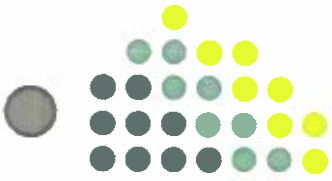
By: Chris Nuhn
November 03, 2009



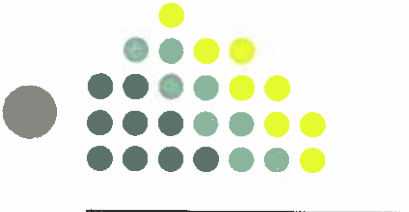
Levels of Optimization

- GCC breaks down optimization into automated levels for quick optimization at the cost of time.
 - -O0
 - -O1
 - -O2
 - -Os
 - -O3
- The default level is -O2, which has no optimization.





Optimization	Included in Level			
	-O1	-O2	-O3	-O4
defer-pop	●	●	●	●
inhibit-jumps	●	●	●	●
branch-probabilities	●	●	●	●
prop-registers	●	●	●	●
guess-branch-probability	●	●	●	●
omit-frame-pointer	●	●	●	●
align-loops	○	●	○	●
align-jumps	○	●	○	●
align-labels	○	●	○	●
align-functions	○	●	○	●
optimize-sibling-calls	○	●	●	●
cse-follow-jumps	○	●	●	●
cse-skip-blocks	○	●	●	●
gcse	○	●	●	●
expensive-optimizations	○	●	●	●
strength-reduce	○	●	●	●
retn-cse-after-loop	○	●	●	●
retn-loop-opt	○	●	●	●
caller-saves	○	●	●	●
force-merit	○	●	●	●
peephole2	○	●	●	●
regmove	○	●	●	●
strict-aliasing	○	●	●	●
delete-null-pointer-checks	○	●	●	●
reorder-blocks	○	●	●	●
schedule-insns	○	●	●	●
schedule-insns2	○	●	●	●
inline-functions	○	○	○	●
rename-registers	○	○	○	●



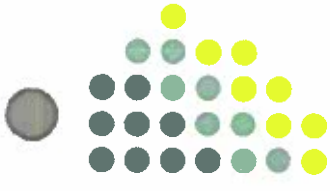
Intermediate Advantages

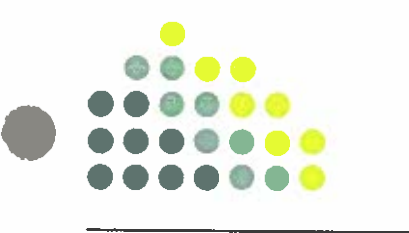
GCC uses a sequence of different intermediate steps to transform code into binary providing many advantages:

- **Breaking code into simpler, lower level languages provides more access for optimization**
- **Intermediate representation allows for multiple language optimization.**
- **Allows for the compiler to use the same mechanics to optimize regardless of what initial language the code was written.**

Intermediate Steps

- With GCC 4.0 and later, the compiler now incorporates GENERIC, GIMPLE, Static Single Assignment (SSA) and Register Transfer Language (RTL)
 1. Source code (any language)
 2. Generic
 3. Gimple
 4. SSA
 5. RTL





Methods Of Optimization

- Common Subexpression Elimination
- Constant Folding
- Copy Propagation Transformation
- Dead Code Elimination
- If-Conversion
- Inlining

Common Subexpression Elimination

```
const int STEP = 3;
const int SIZE = 100;
int i, j, k;
int arr[SIZE]
for(i=0; i < SIZE; i++){
    j = 2 * STEP;
    k = arr[i] * j;
}

const int STEP = 3;
const int SIZE = 100;
int i, j, k;
int arr[SIZE]
j = 2 * STEP;
for(i=0; i < SIZE; i++){
    k = arr[i] * j;
}
```

Constant Folding

```
n = 10 * 20 * 100;
```

```
i = 10;
```

```
j = 20;
```

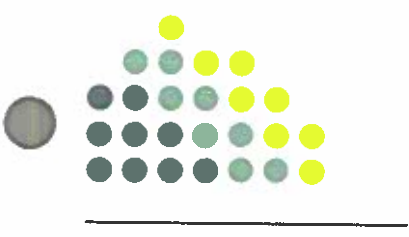
```
ij = i * j;
```

```
n = 200000;
```

```
i = 10;
```

```
j = 20;
```

```
ij = 200;
```





Copy Propagation Transformation

```
i = 10;
```

```
x[j] = i;
```

```
y[MIN] = b;
```

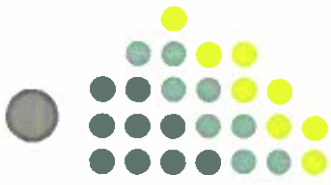
```
b = i;
```

```
i = 10;
```

```
x[j] = 10;
```

```
y[MIN] = b;
```

```
b = 10;
```



Dead Code Elimination

```
if( i == 10){
```

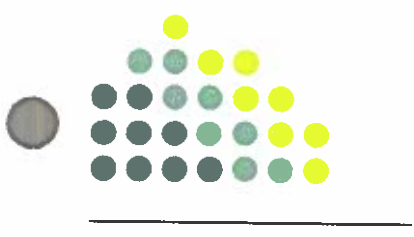
```
}
```

```
else{
```

```
    if(1 == 10){
```

```
    }
```

```
}
```



If-Conversion

```
if(x == 0){
```

```
}
```

```
else if (x == 1){
```

```
}
```

```
else if (x == 2){
```

```
}
```

```
else {
```

```
}
```

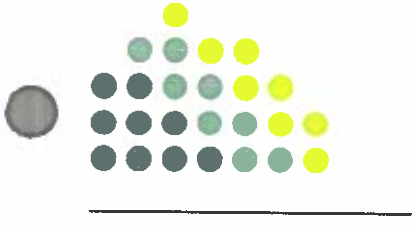
```
switch(x){
```

```
case 1:
```

```
case 2:
```

```
default:
```

```
}
```



Inlining

```
if( y != 0){  
  z = div(x,y);  
}
```

```
div(int x, int y){  
  return x / y;  
}
```

```
if( y != 0){  
  z = x / y;  
}
```

